

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»
Звіт з лабораторної роботи №3
З дисципліни "Моделювання та аналітика даних"
Гradientний спуск для множинної регресії

Виконала:
студентка гр. ПЗМ-25-2
Тимощенко Владислава Русланівна

Перевірів: асистент кафедри ПІ
Дашенков Дмитро Сергійович

Харків 2026

1. Мета: Ознайомитися з реалізацією багатовимірної лінійної регресії методом градієнтного спуску за допомогою NumPy та дослідити вплив різних ознак на точність прогнозування вартості будинків.

2. Хід роботи

2.1 Підготовка даних та get_numpy_data()

Реалізуємо get_numpy_data(dataframe, features, output), яка буде:

- додавати стовпець з одиницями ліворуч від матриці ознак (для вільного члена);
- повертати (feature_matrix, output_array) як масиви NumPy.

```
[4]
✓ Os
def get_numpy_data(dataframe, features, output):
    """
    Будує матрицю ознак NumPy (зі стовпцем вільного члена) та вектор виходу.

    Параметри
    -----
    dataframe : pd.DataFrame
    features   : список str   назви стовпців-ознак
    output     : str          назва стовпця цільової змінної

    Повертає
    -----
    feature_matrix : np.ndarray, shape (n, len(features)+1)
    output_array   : np.ndarray, shape (n,)
    """
    # Add a constant column of 1s (intercept)
    dataframe = dataframe.copy()
    dataframe['constant'] = 1.0

    # Select feature columns and convert to NumPy
    feature_matrix = dataframe[['constant'] + features].to_numpy(dtype=float)

    output_array = dataframe[output].to_numpy(dtype=float)
    return feature_matrix, output_array

▼ Перевірка

[5]
✓ Os
● example_features, example_output = get_numpy_data(sales, ['sqft_living'], 'price')
  print('Перший рядок матриці ознак:', example_features[0, :]) # має бути [1.0, <sqft>]
  print('Перше значення виходу:      ', example_output[0])

  ... Перший рядок матриці ознак: [1.00e+00 1.18e+03]
    Перше значення виходу:      221900.0
```

До набору даних додається стовпець constant, заповнений одиницями, який використовується для обчислення вільного члена моделі. Далі формується матриця ознак із цього стовпця та заданих ознак, після чого вона перетворюється

у масив NumPy. Цільова змінна також перетворюється у одновимірний масив NumPy. Функція повертає матрицю ознак і вектор вихідних значень.

2.2 Реалізація predict_output()

Використаємо один виклик `np.dot` — без явних циклів.

```
▼ Завдання 2 — Реалізація predict_output()

Використовуйте один виклик np.dot — без явних циклів.

[6] ✓ Os
def predict_output(feature_matrix, weights):
    """
    Обчислює передбачення як скалярний добуток кожного рядка з вагами.

    Повертає
    -----
    predictions : np.ndarray, shape (n,)
    """
    predictions = np.dot(feature_matrix, weights)

    return predictions

▼ Перевірка — перші два передбачення мають бути ≈1181 і ≈2571

[7] ✓ Os
my_weights = np.array([1.0, 1.0])
test_preds = predict_output(example_features, my_weights)
print(f'передбачення[0]: {test_preds[0]:.1f} (очікується ≈1181)')
print(f'передбачення[1]: {test_preds[1]:.1f} (очікується ≈2571)')

... передбачення[0]: 1181.0 (очікується ≈1181)
    передбачення[1]: 2571.0 (очікується ≈2571)
```

Функція обчислює передбачені значення моделі за допомогою матричного множення. Для кожного рядка матриці ознак знаходиться скалярний добуток відповідного рядка та вектора ваг `weights`. Результатом є масив прогнозованих значень для всіх об'єктів.

2.3 Реалізація feature_derivative()

Похідна RSS відносно однієї ваги — це $2 \cdot \text{dot}(\text{errors}, \text{feature})$. Реалізуємо як один вираз NumPy.

```
Завдання 3 — Реалізація feature_derivative()

Похідна RSS відносно однієї ваги — це  $2 \cdot \text{dot}(\text{errors}, \text{feature})$ . Реалізуйте як один вираз NumPy.

def feature_derivative(errors, feature):
    """Повертає похідну RSS відносно ваги для даної ознаки."""
    return 2 * np.dot(errors, feature)

Перевірка — похідна відносно константної ознаки має дорівнювати  $-2 \cdot \sum \text{errors}$ 

zero_weights = np.array([0.0, 0.0])
zero_preds = predict_output(example_features, zero_weights)
errors = zero_preds - example_output # = -prices when weights are zero

deriv_constant = feature_derivative(errors, example_features[:, 0])
expected = -2 * np.sum(example_output)
print(f'Обчислено: {deriv_constant:.2e}')
print(f'Очікується: {expected:.2e}')
print(f'Збіг: {np.isclose(deriv_constant, expected)}')
```

```
... Обчислено: -2.33e+10
    Очікується: -2.33e+10
    Збіг: True
```

Функція обчислює похідну суми квадратів залишків (RSS) відносно ваги заданої ознаки. Для цього знаходиться скалярний добуток вектора помилок `errors` та вектора значень ознаки `feature`, після чого результат множиться на 2 відповідно до формули похідної RSS.

2.4 Градієнтний спуск

Реалізуємо `regression_gradient_descent(feature_matrix, output, initial_weights, step_size, tolerance)`. Функція оновлює всі ваги одночасно на кожній ітерації та повертає їх, коли $\|\text{gradient}\|_2 < \text{tolerance}$.

Запустімо з такими параметрами на навчальних даних:

```
features = ['sqft_living']
initial_weights = [-47000., 1.]
step_size = 7e-12
tolerance = 2.5e7
```

Вкажіть навчені ваги. Яка передбачувана ціна першого будинку в **тестовій** вибірці? Обчисліть RSS на всій тестовій вибірці.

```
[14] ● def regression_gradient_descent(feature_matrix, output,
    ✓ Os      initial_weights, step_size, tolerance):
    """
    Мінімізує RSS за допомогою пакетного градієнтного спуску.

    Повертає
    -----
    weights : np.ndarray — навчені ваги
    """
    weights = np.array(initial_weights, dtype=float)
    converged = False

    while not converged:
        # 1. Compute predictions
        predictions = predict_output(feature_matrix, weights)

        # 2. Compute errors
        errors = predictions - output

        gradient_sum_squares = 0.0
        for i in range(len(weights)):
            # 3. Derivative for weight i
            derivative = feature_derivative(errors, feature_matrix[:, i])

            # 4. Accumulate squared gradient
            gradient_sum_squares += derivative ** 2

            # 5. Update weight
            weights[i] -= step_size * derivative

        # 6. Check convergence
        gradient_magnitude = sqrt(gradient_sum_squares)
        if gradient_magnitude < tolerance:
            converged = True

    return weights
```

▼ Запуск Моделі 1 — одна ознака

```
[15]
✓ Os
simple_feature_matrix, output = get_numpy_data(train_data, ['sqft_living'], 'price')

simple_weights = regression_gradient_descent(
    simple_feature_matrix, output,
    initial_weights=[-47000., 1.],
    step_size=7e-12,
    tolerance=2.5e7
)
print('Навчені ваги:', simple_weights)
```

Навчені ваги: [-46999.88018846 280.51005013]

▼ Оцінка Моделі 1 на тестовій вибірці

```
[16] ● test_feature_matrix, test_output = get_numpy_data(test_data, ['sqft_living'], 'price')
✓ Os

# Predicted price for the first test house
test_predictions = predict_output(test_feature_matrix, simple_weights)
print(f'Передбачена ціна (будинок 0): ${test_predictions[0]:.0f}')
print(f'Реальна ціна (будинок 0): ${test_output[0]:.0f}')

# RSS on the full test set
rss_model1 = np.sum((test_predictions - test_output) ** 2)
print(f'Тестова RSS Моделі 1: {rss_model1:.3e}')
```

... Передбачена ціна (будинок 0): \$354,129
Реальна ціна (будинок 0): \$297,000
Тестова RSS Моделі 1: 1.204e+15

В кожній ітерації градієнтного спуску обчислюються прогнозовані значення моделі та помилки прогнозування. Для кожної ваги знаходиться похідна

функції втрат RSS, після чого вага оновлюється у напрямку, протилежному градієнту. Одночасно накопичується сума квадратів похідних для обчислення норми градієнта. Навчання завершується, коли величина градієнта стає меншою за заданий поріг `tolerance`.

Яка передбачувана ціна першого будинку в тестовій вибірці? Обчисліть RSS на всій тестовій вибірці.

Модель оцінила вартість першого будинку у 354 129 доларів, тоді як його фактична ціна становить 297 000 доларів. Значення RSS на тестовій вибірці дорівнює 1.204×10^{15} та характеризує сумарну похибку прогнозів моделі для всіх об'єктів тестового набору.

2.5 Бонус — Додайте другу ознаку

Повторимо градієнтний спуск з `['sqft_living', 'sqft_living15']` та параметрами:

```
initial_weights = [-100000., 1., 1.]
```

```
step_size      = 4e-12
```

```
tolerance      = 1e9
```

Обчислимо RSS на тестовій вибірці та порівняйте з моделлю з однією ознакою. Порівняймо передбачену ціну першого тестового будинку з обох моделей з реальною ціною.

```

[17]
✓ Os
● # Бонус – модель з двома ознаками
# Параметри: initial_weights=[-100000., 1., 1.], step_size=4e-12, tolerance=1e9
train_feature_matrix, train_output = get_numpy_data(
    train_data,
    ['sqft_living', 'sqft_living15'],
    'price'
)

two_feature_weights = regression_gradient_descent(
    train_feature_matrix,
    train_output,
    initial_weights=[-100000., 1., 1.],
    step_size=4e-12,
    tolerance=1e9
)

print('Навчені ваги:', two_feature_weights)

test_feature_matrix, test_output = get_numpy_data(
    test_data,
    ['sqft_living', 'sqft_living15'],
    'price'
)

test_predictions_2 = predict_output(
    test_feature_matrix,
    two_feature_weights
)

rss_model2 = np.sum((test_predictions_2 - test_output) ** 2)

print(f'Передбачена ціна (Модель 2): ${test_predictions_2[0]:,.0f}')
print(f'Реальна ціна: ${test_output[0]:,.0f}')
print(f'Тестова RSS Моделі 2: {rss_model2:.3e}')

print(f'Тестова RSS Моделі 1: {rss_model1:.3e}')

▼ ... Навчені ваги: [-9.99998826e+04  2.43610628e+02  6.54855462e+01]
Передбачена ціна (Модель 2): $342,008
Реальна ціна: $297,000
Тестова RSS Моделі 2: 1.186e+15
Тестова RSS Моделі 1: 1.204e+15

```

Створюється модель лінійної регресії з двома ознаками: `sqft_living` та `sqft_living15`. Навчання виконується методом градієнтного спуску із заданими початковими вагами, кроком навчання та критерієм зупинки. Після навчання обчислюються прогнозовані значення для тестової вибірки, визначається RSS та виконується порівняння з моделлю, яка використовує лише одну ознаку.

Додавання другої ознаки дозволило моделі враховувати більше інформації про нерухомість, тому точність прогнозів дещо покращилася. Це

підтверджується як зменшенням RSS, так і більш точним прогнозом вартості першого будинку тестової вибірки.

3. Висновки: У ході роботи було реалізовано основні етапи побудови лінійної регресії без використання готових моделей scikit-learn: формування матриці ознак, обчислення прогнозів, похідних та навчання методом градієнтного спуску. Було побудовано модель для прогнозування вартості будинків за ознакою `sqft_living` та оцінено її якість за допомогою показника RSS. Додавання ознаки `sqft_living15` дозволило зменшити RSS з 1.204×10^{15} до 1.186×10^{15} , що свідчить про покращення точності прогнозування завдяки використанню додаткової інформації.

4. Покликання

Репозиторій GitHub: [покликання](#)

Відео на YouTube: [покликання](#)